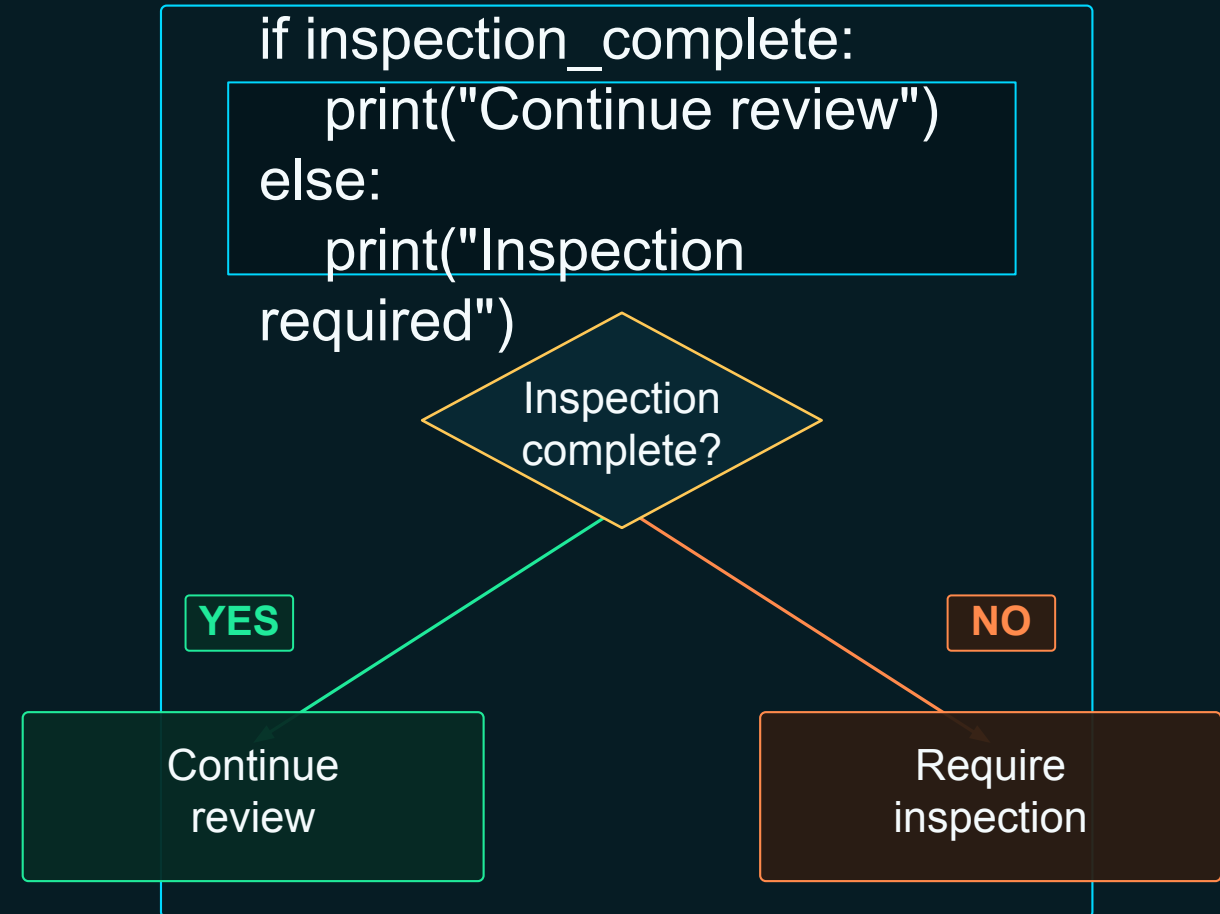


MAKING DECISIONS WITH IF AND ELSE

Block 13 • Conditional execution, code paths, and beginner-friendly tracing



The big idea: a program can choose a path

Before if statements

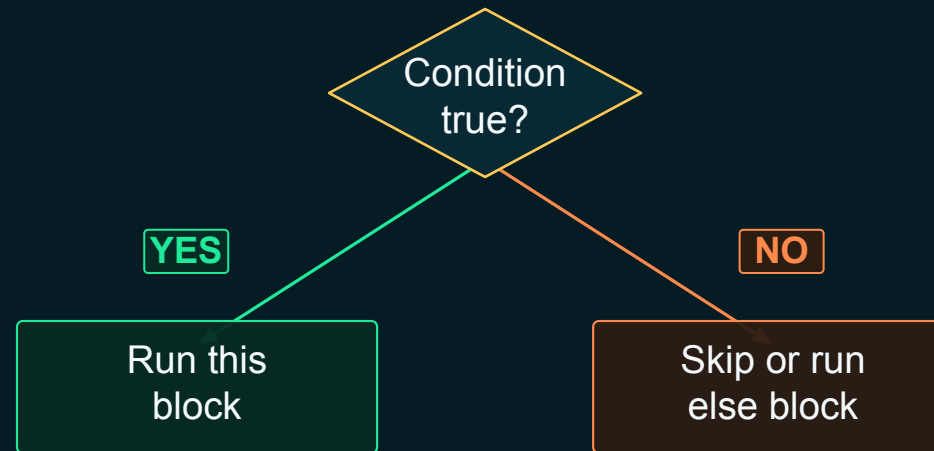
Every line ran in order.
The program was a straight road.

Now

A Boolean condition can decide
which block of code runs.

Still true under the hood

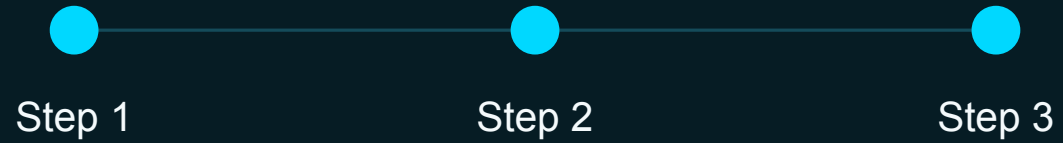
The processor executes
instructions one at a time.
The branch chooses which
instructions come next.



Conditionals are where a program starts to feel intelligent: it reacts to data.

Sequential execution: the straight road

```
print("Step 1")  
print("Step 2")  
print("Step 3")
```



What happens?

Python starts at the top and executes each statement.

No question is asked. No path is skipped.

Mental model

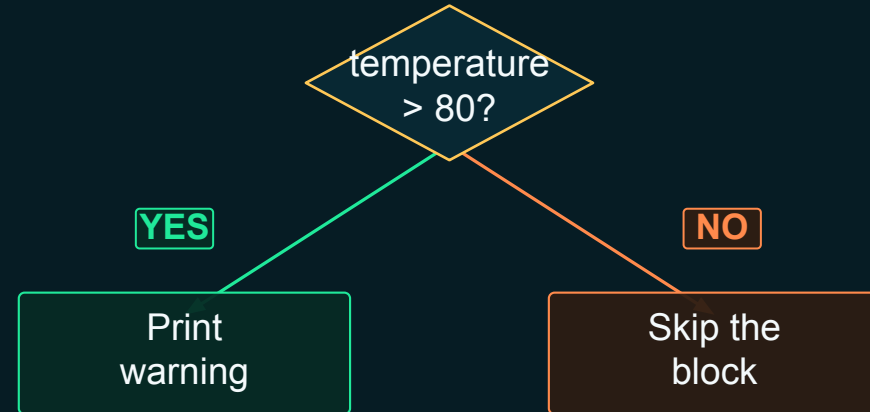
This is like following a recipe where every instruction is mandatory.

Most code is still sequential; conditionals simply add controlled branches.

Conditional execution: ask a question first

```
temperature = 85
```

```
if temperature > 80:  
    print("Temperature warning")
```



The condition

`temperature > 80` produces a Boolean result: True or False.

The block

The indented line belongs to the if statement.

The outcome

If the condition is True, the block runs. Otherwise it is skipped.

An if statement does not create magic. It creates a branch in the instruction path.

Anatomy of a basic if statement

```
if condition:  
    code_that_runs_when_true
```

1. **if keyword**

Tells Python a conditional statement is starting.

2. **condition**

An expression that becomes True or False.

3. **colon**

Marks the start of a code block.

4. **indentation**

Shows which lines belong inside the block.

Syntax matters because Python uses it to understand your intent.

Conditions are Boolean expressions

```
score = 78  
score >= 70    # True
```

```
file_name = "equipment.csv"  
file_name.endswith(".csv") # True
```

Comparison

```
score >= 70
```

String method

```
file_name.endswith(".csv")
```

Stored Boolean

```
is_ready = True
```

Combined logic

```
A and B and not C
```

```
if score >= 70:  
    print("Pass")
```

```
if file_name.endswith(".csv"):  
    print("Supported")
```

Anything that evaluates to True or False can control an if statement.

Code blocks: colons and indentation

```
if temperature > 80:  
    print("Warning")  
    print("Check cooling system")
```

Both indented lines run only when true

The block can contain one statement or many statements.

```
if temperature > 80  
    print("Warning")
```

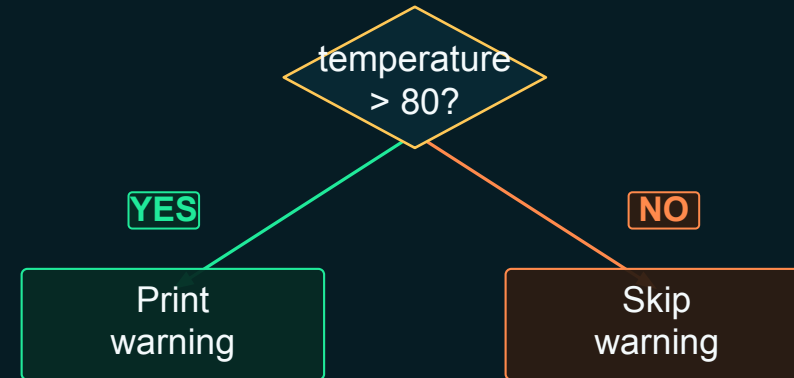
Missing colon

Python does not know where the block begins.

The colon says “a block is coming.” Indentation says “this line belongs to the block.”

Indentation changes what runs

```
if temperature > 80:  
    print("Warning")  
  
print("Program complete")
```



Indented line

Runs only if the condition is True.

Unindented line

Runs after the if statement, regardless of the condition.

Trace habit

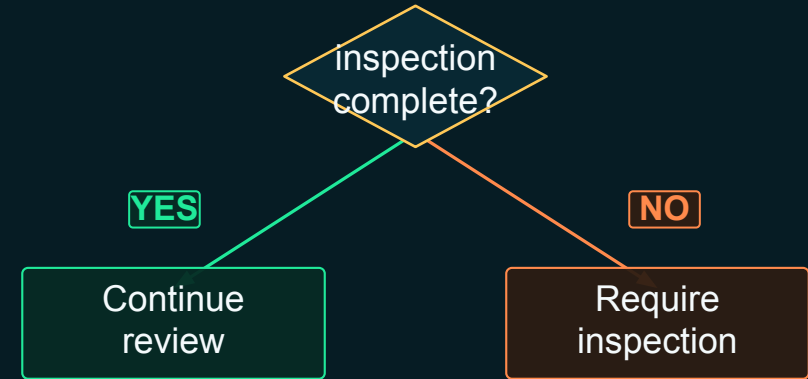
Ask: which lines are inside the block?

When tracing, draw a box around the indented block.

Two-way decisions with else

```
inspection_complete = True

if inspection_complete:
    print("Equipment may continue to
    readiness review.")
else:
    print("Inspection is required.")
```



Exactly one path runs

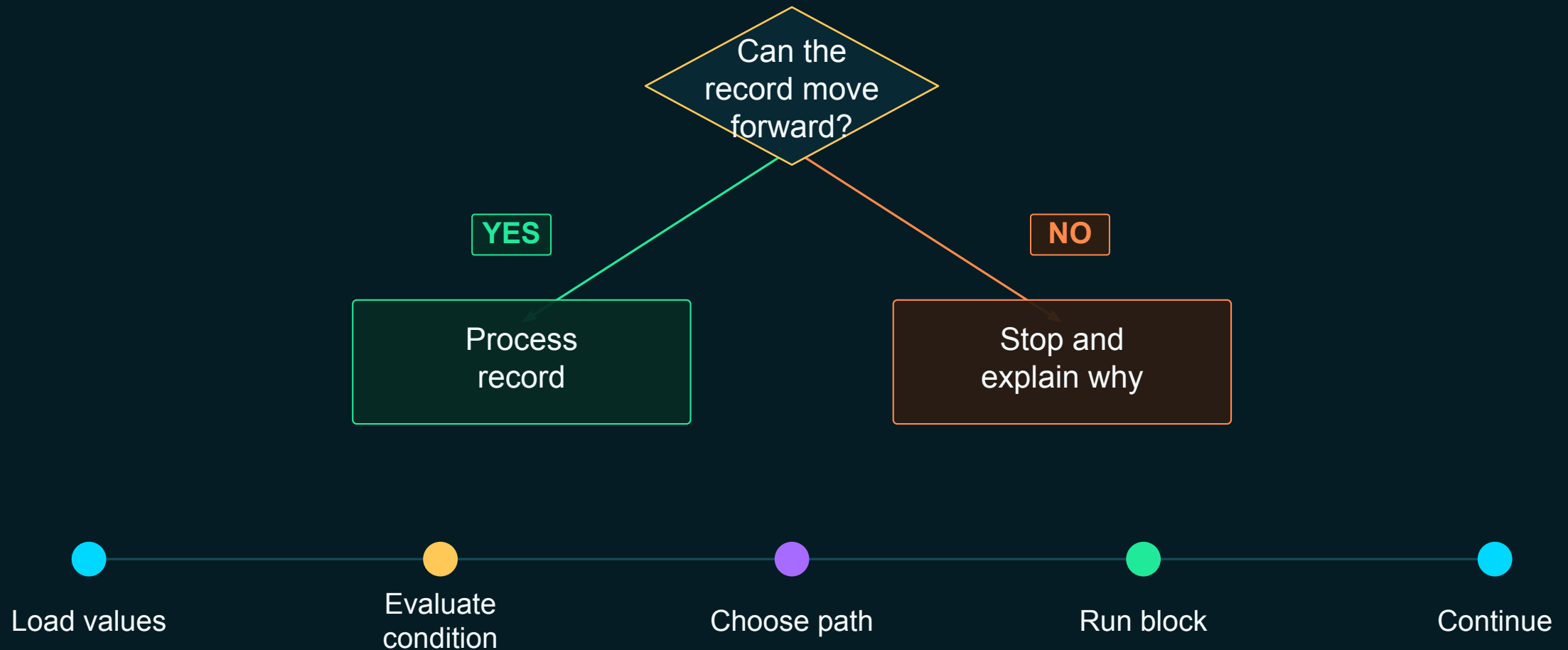
If the condition is True, the if block runs.
If it is False, the else block runs.

else has no condition

It means "otherwise." It catches the other path.

if / else is a fork in the road: one branch or the other.

Execution paths: the branch mental model



The processor still runs one instruction at a time; the condition changes which sequence comes next.

Stored Boolean or direct comparison?

```
is_within_limit = operating_hours <=  
maximum_hours
```

```
if is_within_limit:  
    print("Within operating limit")
```

```
if operating_hours <=  
maximum_hours:  
    print("Within operating limit")
```

Stored Boolean

Good when the condition has business meaning or will be reused.

Direct condition

Good when the comparison is short and obvious.

Beginner tip: naming important conditions often makes code easier to trace.

Manual tracing: predict the path before running

```
score = 78
```

```
if score >= 70:  
    print("Pass")  
else:  
    print("Fail")
```

```
print("Done")
```

Trace checklist

1. List current variable values
2. Evaluate the condition
3. Mark True or False
4. Run only the matching block
5. Continue after the branch

Prediction: Pass, then Done

Do not just read code. Execute it in your head, one line at a time.

Common syntax and indentation errors

```
if score >= 70  
    print("Pass")
```

Missing colon

The if line must end with
`:`.

```
if score >= 70:  
print("Pass")
```

Missing indentation

The block must be
indented.

```
if score >= 70:  
    print("Pass")  
    print("Extra")
```

Uneven indentation

Indent consistently inside
the block.

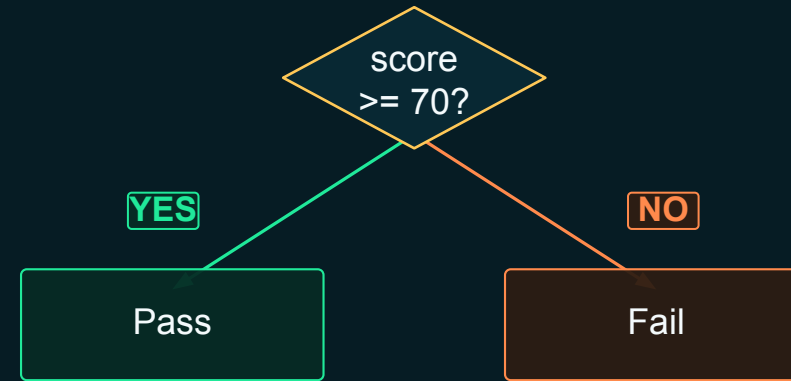
Debug habit: read the final line of the error message, then inspect the line above it too.

Syntax errors are not personal. They are the interpreter asking for clearer instructions.

Instructor demo: pass or fail

```
score = 78
```

```
if score >= 70:  
    print("Pass")  
else:  
    print("Fail")
```



Change the data

Try score = 69, then score = 70.
Ask students which path will run.

Key habit

The structure stays the same.
Only the input value changes the path.

The same program can produce different outputs for different data.

Guided exercise 1: Pass or fail

```
score = 78

if score >= 70:
    print("Pass")
else:
    print("Fail")
```

Your tasks

1. Run the code.
2. Change the score.
3. Predict before running.
4. Test score values 69, 70, and 100.

Question: which score is the first passing score?

Predict → run → compare. That loop builds programming intuition.

Exercise 1 answer: trace the boundary

score = 69

69 >= 70 → False

Output: Fail

score = 70

70 >= 70 → True

Output: Pass

score = 100

100 >= 70 → True

Output: Pass

```
if score >= 70:  
    print("Pass")  
else:  
    print("Fail")
```

The boundary value matters: because the operator is >=, 70 passes.

Guided exercise 2: Inventory status + file type

```
quantity_on_hand = 14
reorder_level = 20

if quantity_on_hand <= reorder_level:
    print("Reorder required")
else:
    print("Inventory level acceptable")
```

```
file_name = "equipment.csv"

if file_name.lower().endswith(".csv"):
    print("Supported file type")
else:
    print("Unsupported file type")
```

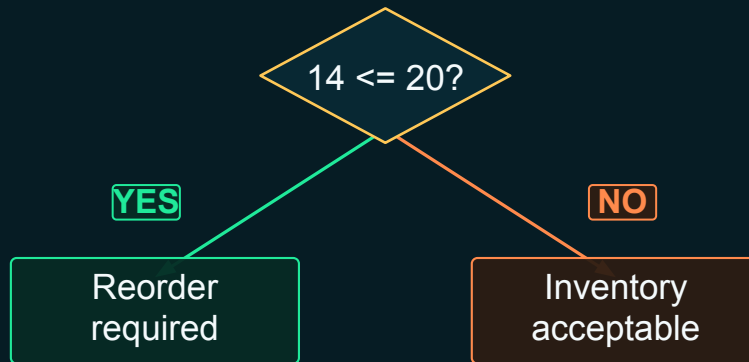
Think first

Inventory: is 14 less than or equal to 20?

File: does the normalized file name end with .csv?

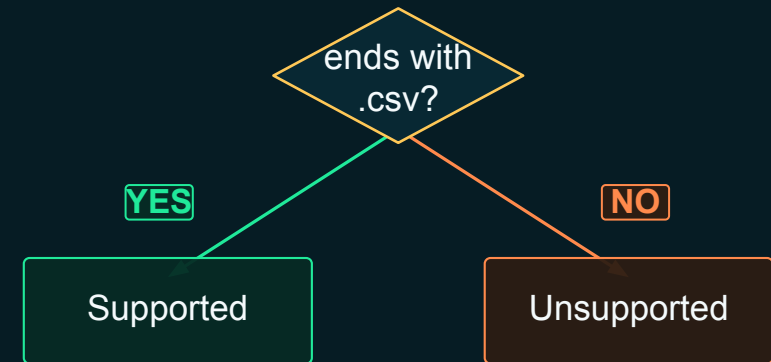
Same if/else structure; different kind of condition.

Exercise 2 answer: different data, same decision shape



Inventory output

Reorder required



File output

Supported file type

Notice the verbs: compare, normalize, then decide.

Guided exercise 3: Approval status

```
approval_status = " approved "  
approval_status =  
approval_status.strip().lower()  
  
if approval_status == "approved":  
    print("Processing authorized")  
else:  
    print("Processing not authorized")
```

Why clean first?

Without `.strip()`, the extra spaces would make the comparison fail.

What is the condition?

`approval_status == "approved"` produces `True` or `False`.

Normalize text before comparing it. Data from humans is rarely perfectly typed.

Predict the path: do not run it yet

```
system_locked = False
training_complete = True

if training_complete and not system_locked:
    print("Ready")
else:
    print("Blocked")

print("Check complete")
```

Student questions

1. Which condition is evaluated?
2. Is it True or False?
3. Which statements run?
4. What exact output appears?

Write your prediction before pressing Run.

Tracing is a skill. Practice it slowly while the programs are still small.

Predict-the-path answer

Condition

training_complete and not
system_locked

Values

True and not False
True and True

Result

Condition is True

Ready
Check complete

The final print is not indented, so it runs after either path.

Applied challenge: access decision program

Store these values

has_account
account_active
training_complete
system_locked

Access is granted only if...

User has an account
Account is active
Training is complete
System is not locked

Output

Access granted
or
Access denied

```
can_access = (  
    has_account  
    and account_active  
    and training_complete  
    and not system_locked  
)
```

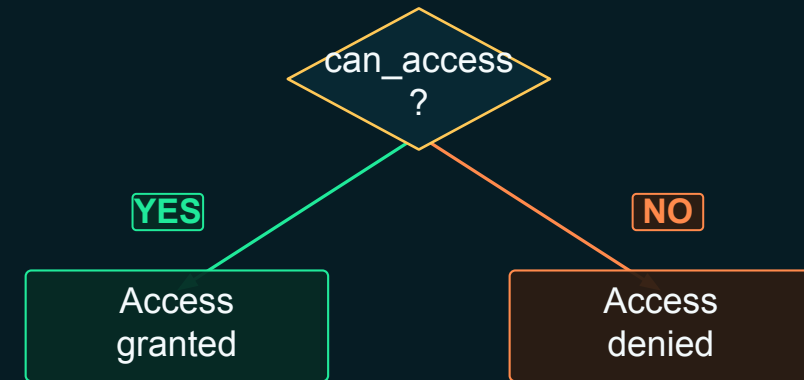
Build the decision in small parts, then combine the parts.

Applied challenge version 1: combined condition

```
has_account = True
account_active = True
training_complete = True
system_locked = False
```

```
can_access = (
    has_account
    and account_active
    and training_complete
    and not system_locked
)
```

```
if can_access:
    print("Access granted")
else:
    print("Access denied")
```



Try changes

Set one value to False.
Set `system_locked` to True.
Predict each output.

This works, but it does not explain why access was denied.

Applied challenge version 2: explain every check

```
print(f"Has account: {has_account}")
print(f"Account active: {account_active}")
print(f"Training complete:
{training_complete}")
print(f"System locked: {system_locked}")
print(f"Can access: {can_access}")
```

Why print intermediate checks?

Users and developers need to understand decisions.
Good code is not only correct. It is explainable.

```
if can_access:
    print("Access granted")
else:
    print("Access denied")
```

Print the individual facts first, then print the final decision.

Knowledge check: can you explain it plainly?

Question 1

What does an if statement do?

Question 2

Why does indentation matter?

Question 3

What does else mean?

Question 4

What is the difference between a condition and a block?

Question 5

Why might we store a condition in a Boolean variable?

Question 6

How do you trace which path runs?

Next step: more than two paths with elif, then loops that repeat decisions.